

Comprehensive Code Audit Report

Skill Stack Architecture Blueprint

Next.js 16 + TypeScript + Tailwind CSS 4 + Framer Motion

Audit Date	2026-06-08
Auditor	Senior Code Auditor (Automated)
Project Root	/home/z/my-project/
Framework	Next.js 16.1.1 (App Router)
Total Source Files	18 custom + 40 shadcn/ui
Lines of Custom Code	~2,200 lines

1. Executive Summary

This audit evaluates the Skill Stack Architecture Blueprint project, a Next.js 16 application built with TypeScript, Tailwind CSS 4, Framer Motion, and Recharts. The project renders an interactive, editorial-style web experience showcasing 16 skills across 4 tiers, a 7-dimension design algorithm (SP-7), 5 design options, an AI portal gateway with intent classification, and a comparative analysis radar chart. The audit covered all custom source files, configuration files, and supporting infrastructure code, examining syntax and structural correctness, logic and runtime behavior, security posture, adherence to best practices and style conventions, and edge-case robustness. Below is a high-level health assessment followed by a detailed findings table.

Category	Rating	Findings	Assessment
Syntax / Structure	LOW	1	Minor TS property typo in one file
Logic / Correctness	MEDIUM	5	Weight normalization, infinite scroll, stale closures, lightbox keys
Security	HIGH	4	CORS wildcard, clipboard without fallback, unsanitized input, Caddy proxy open
Best Practices / Style	MEDIUM	7	Disabled lint rules, ignored TS errors, hardcoded colors, no error boundaries
Edge Cases / Robustness	MEDIUM	4	Empty search, zero maxScore, lightbox close, mobile hamburger keyboard

The project is architecturally sound with a clean component hierarchy, proper use of Next.js App Router patterns, and well-structured data models. The editorial design system with Ink and Vermillion palette is consistently applied. However, several security and robustness issues require attention before production deployment, and the deliberately disabled linting and TypeScript strictness settings create risk for future maintenance. The total finding count is 21 across 5 categories, with 4 HIGH-severity security items that should be addressed as a priority.

2. Detailed Findings

Each finding is categorized by severity, mapped to the exact file and line number, accompanied by a description of the issue, and paired with a recommended fix. Severity levels follow the CVSS-inspired scale: CRITICAL (immediate exploitation risk), HIGH (significant security or correctness impact), MEDIUM (degraded quality or maintainability), LOW (minor style or convention deviation), and INFO (observations for awareness).

S01 | HIGH | Caddyfile:8

Issue: Open reverse proxy: Caddy accepts XTransformPort from any query parameter and proxies to any localhost port. An external attacker can craft URLs with arbitrary port values to access internal services running on the host, including databases, admin panels, or other microservices. This creates a Server-Side Request Forgery (SSRF) vector on localhost.

Fix: Restrict XTransformPort to a whitelist of allowed ports (e.g., 3000, 3003, 4100, 8082, 20128). Validate the port value against the whitelist before proxying. Remove the wildcard query parameter handler or add authentication middleware.

S02 | **HIGH** | examples/websocket/server.ts:8-9

Issue: CORS configured with origin: "*" allows any website to connect to the WebSocket server. Combined with the Caddy proxy SSRF, this means any external domain can establish a WebSocket connection, read messages, and impersonate users.

Fix: Replace wildcard origin with an explicit list of allowed domains (e.g., your production domain and localhost for development). Add token-based authentication on the join event.

S03 | **HIGH** | src/components/SkillReference.tsx:34

Issue: navigator.clipboard.writeText() is called without a try-catch block. This API is only available in secure contexts (HTTPS) and requires user activation. It will throw in HTTP environments, older browsers, or when the page lacks focus, causing an unhandled promise rejection that crashes the copy interaction silently.

Fix: Wrap in try-catch with a fallback using document.execCommand("copy") or display an error toast. The ImplementationBlueprint.tsx component (line 94) has the same issue.

S04 | **HIGH** | src/components/AIPortalGateway.tsx:48-51

Issue: User input from the search field is used directly in keyword matching without any sanitization or rate limiting. While the current implementation only performs client-side string matching (no injection risk), the search loop iterates over all intent routes for every keystroke without debouncing, causing unnecessary re-renders and CPU usage on rapid typing.

Fix: Add a debounce (300ms) to the handleSearch function using setTimeout/clearTimeout or a custom hook. This also prepares the architecture for future server-side search where injection would be a real concern.

L01 | **MEDIUM** | src/components/DesignAlgorithm.tsx:82-85

Issue: The SP-7 weight sliders allow values from 0-50% per dimension, but the weights are never normalized. If a user sets all 7 sliders to 50%, the total weight sums to 350% instead of 100%. The computeScore function on line 12-14 computes a weighted sum that can produce arbitrarily large scores, making the prioritized results meaningless for extreme slider configurations.

Fix: Normalize the customWeights array before computing scores: divide each weight by the sum of all weights. Alternatively, constrain the slider interaction so that adjusting one slider proportionally reduces others.

L02 | MEDIUM | src/components/DesignAlgorithm.tsx:21

Issue: `Math.max(...scoredOptions.map(o => o.score))` will return `-Infinity` if `scoredOptions` is empty (though unlikely with static data). More importantly, if all weights are set to 0, all scores become 0, and `maxScore` becomes 0, causing division by zero in the bar width calculation on line 134: `(opt.score / maxScore) * 100` produces `NaN`.

Fix: Add a guard: `const maxScore = Math.max(0.001, Math.max(...scoredOptions.map(o => o.score)))`; This ensures the bar width calculation never produces `NaN`.

L03 | MEDIUM | src/components/VisualGallery.tsx:94,124

Issue: The lightbox component does not use `AnimatePresence`, so exit animations never fire. Additionally, the lightbox key is based on `selectedImage` state which can be "editorial" but there is no matching entry in `optionImages`, so the lightbox renders without an image src when the editorial image is clicked. The selected variable on line 18 uses `optionImages.find()` which returns undefined for "editorial".

Fix: Wrap the lightbox in `AnimatePresence`. Add the editorial image to `optionImages` array or handle the "editorial" case separately with a dedicated lightbox component for the full-width image.

L04 | MEDIUM | src/components/DecisionTree.tsx:202-206

Issue: The Go Back button on line 202-206 slices the path array and sets `currentNode` based on whether the previous path entry exists in `decisionTree`. However, if the user goes back from a result node (result-*), the code checks `decisionTree[prev]` which will be undefined for result nodes, causing the fallback to "start" instead of the correct previous question node.

Fix: When going back from a result, also clear the result state. Check if the previous node is a result node and handle accordingly: `if (results[prev]) { setResult(null); } setCurrentNode(prev in decisionTree ? prev : "start");`

L05 | MEDIUM | src/app/page.tsx:34-36

Issue: The `scrollTo` function uses `document.getElementById(id)?.scrollIntoView()` but does not account for the sticky navigation bar (`h-12 = 48px`). Scroll targets will be obscured by the fixed navbar. Additionally, this is a client-side smooth scroll without any URL hash update, so deep linking to sections is not supported.

Fix: Use `scrollIntoView({ behavior: "smooth", block: "start" })` with a `scroll-margin-top` CSS offset on each section target, or manually calculate the scroll position: `window.scrollTo({ top: el.offsetTop - 60, behavior: "smooth" })`. Update `window.location.hash` for deep linking support.

B01 | MEDIUM | next.config.ts:7-9

Issue: `typescript.ignoreBuildErrors: true` and `reactStrictMode: false` are set in the Next.js configuration. The first setting silently allows type errors to pass through production builds, hiding bugs that TypeScript would normally catch. The second disables React strict mode which suppresses important development warnings about deprecated lifecycle methods and unsafe patterns.

Fix: Set `ignoreBuildErrors`: false and fix any type errors that surface. Enable `reactStrictMode`: true. These are development aids that prevent production regressions.

B02 | MEDIUM | `eslint.config.mjs`:9-44

Issue: Over 20 ESLint rules are explicitly set to "off", including `@typescript-eslint/no-explicit-any`, `@typescript-eslint/no-unused-vars`, `react-hooks/exhaustive-deps`, and `no-console`. This effectively disables most of the value that linting provides. The project has no meaningful lint enforcement.

Fix: Re-enable critical rules incrementally. Start with: `no-unused-vars` (warn), `no-explicit-any` (warn), `exhaustive-deps` (error), `no-console` (warn). Fix violations as they appear rather than suppressing them globally.

B03 | MEDIUM | `tsconfig.json`:13

Issue: `noImplicitAny`: false is set in `tsconfig.json`, allowing variables and parameters to have implicit any type. This defeats the purpose of TypeScript type safety and can mask runtime type errors. Combined with `ignoreBuildErrors` in `next.config.ts`, the project has no type enforcement at compile time.

Fix: Set `noImplicitAny`: true and add explicit type annotations where needed. This is a foundational TypeScript setting that should be enabled for any production project.

B04 | MEDIUM | `src/components/ComparativeAnalysis.tsx`:8-13

Issue: The `radarData` computation at module level (outside the component) iterates over options on every module import. The `optColors` array on line 15 uses hardcoded hex values that duplicate the CSS variable values from `globals.css`. If the design palette changes, these hardcoded colors must be updated separately, creating a maintenance drift risk.

Fix: Move the `radarData` computation inside the component (or use `useMemo`) so it only runs when the component renders. Import colors from a shared constants file or reference CSS variables via `getComputedStyle` to maintain single-source-of-truth for the palette.

B05 | MEDIUM | `src/components/HeatmapViz.tsx`:57-59

Issue: Heatmap cell colors use hardcoded `rgba(194, 54, 22, ...)` which is the raw RGB equivalent of the CSS primary color `#C23616`. If the theme changes (e.g., dark mode toggle), these colors will not adapt. The component also hardcodes the text colors `"#C23616"` and `"#6B6B6B"` inline.

Fix: Use CSS custom properties via `style={{ backgroundColor: "var(--primary)" }}` with opacity control, or use the `tailwind-merge` utility with primary color classes. This ensures theme consistency and dark mode support.

B06 | MEDIUM | `src/lib/skill-data.ts`:229

Issue: The `heatmapData` for section 12 (AI Portal Gateway) has 12 values per row, matching the 12 section labels. However, the `SectionMapping` component defines 12 section entries (`sec01-sec12`) but section `sec12` has both `sp9Score` (typo) and `sp7Score` properties on line 29. The `sp9Score` property does not match the `SP7Dimension` interface and will cause TypeScript confusion if strict mode is enabled.

Fix: Remove the sp9Score property from section sec12. It appears to be a leftover from a previous iteration that renamed SP-9 to SP-7. Only sp7Score should remain.

B07 | **LOW** | src/app/layout.tsx:25-35

Issue: The OpenGraph and Twitter metadata titles and descriptions reference "Z.ai Code Scaffold" and "AI-powered development" rather than the actual project name "Skill Stack Architecture". This creates misleading social media previews when the page is shared.

Fix: Update the openGraph and twitter metadata to match the actual page title and description already defined in the metadata object: "Skill Stack Architecture | Editorial" and the existing description string.

E01 | **LOW** | src/components/AIPortalGateway.tsx:13-21

Issue: When the search query contains a keyword that matches multiple routes equally (e.g., "design" matches both "Design Philosophy" and "Design Algorithm" keywords), the algorithm picks whichever route appears last in the loop. This is because the > comparison on line 19 does not break ties deterministically.

Fix: When scores are equal, apply a secondary tiebreaker such as route.confidence (higher confidence wins) or the order in the intentRoutes array (first match wins). This ensures consistent routing for ambiguous queries.

E02 | **LOW** | src/components/VisualGallery.tsx:129

Issue: The lightbox overlay uses onClick={() => setSelectedImage(null)} on the backdrop, which also triggers for mouse events that originate on interactive children. While e.stopPropagation() is used on line 135, this pattern can cause issues with screen readers and keyboard navigation. There is no Escape key handler to close the lightbox.

Fix: Add a useEffect that listens for the Escape key and calls setSelectedImage(null). Add an aria-label to the close button and ensure focus trapping within the lightbox for accessibility compliance.

E03 | **LOW** | src/app/page.tsx:79-94

Issue: The mobile menu uses conditional rendering ({mobileMenuOpen && ...}) without any transition animation. When the menu opens or closes, it appears and disappears instantly, which is jarring for users. Additionally, the hamburger icon animation uses basic CSS transforms that do not account for the center bar disappearing, creating a visual glitch.

Fix: Use AnimatePresence from framer-motion with initial/exit animations for the mobile menu panel. Refine the hamburger animation to properly cross the top and bottom bars while fading the center bar.

E04 | **LOW** | src/app/globals.css:126-135

Issue: The editorial-dropcap::first-letter pseudo-element applies float:left with font-size 3.5rem, which can cause layout issues in narrow containers or when the first paragraph is short. The drop cap may overlap subsequent content or push the paragraph text into an awkward narrow column.

Fix: Add a max-width constraint or use a fixed-width span for the drop cap instead of float. Test with paragraphs of varying length, especially on mobile viewports where the content area is narrower.

I01 | [INFO](#) | package.json:61-80

Issue: Several production dependencies are imported but never used in the custom source code: next-auth, react-hook-form, @dnd-kit/core, @mdxeditor/editor, zustand, next-intl, next-themes, sharp, and z-ai-web-dev-sdk. These add to the bundle size and increase install time. The @prisma/client is imported in db.ts but the Prisma schema (User/Post models) is unrelated to the skill stack application.

Fix: Audit each dependency for actual usage. Remove unused packages to reduce the node_modules size and build time. If these are scaffolding for future features, document them as planned dependencies in a README section.

3. Configuration and Infrastructure Audit

3.1 Next.js Configuration (next.config.ts)

The Next.js configuration sets output: "standalone" which is correct for containerized deployments. However, the typescript.ignoreBuildErrors: true setting is a significant concern. This means the production build pipeline completely bypasses TypeScript type checking, allowing type errors to silently enter production. The reactStrictMode: false setting disables React strict mode, which suppresses important development warnings about unsafe lifecycle methods, legacy context API usage, and unexpected side effects. Both settings should be reconsidered for a production-ready project. The standalone output mode correctly copies static assets and public files in the build script, which is the proper pattern for Docker deployments of Next.js applications.

3.2 TypeScript Configuration (tsconfig.json)

The tsconfig.json sets strict: true but then immediately undermines it with noImplicitAny: false. This is contradictory: strict mode enables noImplicitAny by default, but the explicit override disables it. The target ES2017 is appropriate for server-side rendering but may exclude some modern browser APIs if client-side code uses them. The moduleResolution: "bundler" setting is correct for Next.js 16 which uses Turbopack. The path alias @/* mapping to ./src/* is properly configured for clean imports throughout the project.

3.3 ESLint Configuration (eslint.config.mjs)

The ESLint configuration disables over 20 rules across TypeScript, React, and general JavaScript categories. While this reduces noise during development, it effectively eliminates the value of having a linter at all. The most concerning disabled rules are: @typescript-eslint/no-explicit-any (allows untyped code), react-hooks/exhaustive-deps (allows stale closure bugs), and @typescript-eslint/no-unused-vars (allows dead code accumulation). The ignores list correctly excludes node_modules, .next, and examples directories, but also ignores the skills directory which may contain code that should be linted if

it is part of the deployed application.

3.4 Tailwind CSS Configuration (tailwind.config.ts)

The Tailwind configuration file still uses the v3 format (darkMode: "class", content array, theme.extend with hsl() color functions) despite the project using Tailwind CSS v4 (as indicated in package.json with "^4"). Tailwind v4 uses a CSS-first configuration approach via the @theme directive in globals.css, which the project already has. This tailwind.config.ts file is therefore redundant and potentially conflicting. The hsl() wrapping of CSS variables in the config is unnecessary because the globals.css already defines the variables as raw hex values rather than HSL components. This mismatch could cause color rendering issues or build warnings in Tailwind v4.

3.5 Prisma Schema (schema.prisma)

The Prisma schema defines User and Post models with a SQLite datasource, but these models are entirely unrelated to the Skill Stack Architecture application. The db.ts file creates a PrismaClient singleton with query logging enabled, which is appropriate for development but should be reduced to error-only logging in production. The DATABASE_URL environment variable is expected but there is no .env.example file documenting its required value. The schema lacks any migration files, suggesting the database has not been properly versioned. If the User/Post models are scaffolding for future features, they should be documented as such; otherwise they add unnecessary complexity and bundle size through @prisma/client.

3.6 Caddy Reverse Proxy (Caddyfile)

The Caddyfile implements a reverse proxy on port 81 that defaults to localhost:3000 (the Next.js app) but also accepts an XTransformPort query parameter to proxy to any localhost port. This design is intended for development convenience, allowing access to multiple services through a single entry point. However, the unrestricted port proxying creates a significant security vulnerability: any external request can access any service running on the host by specifying the appropriate port number. This includes databases (e.g., PostgreSQL on 5432, Redis on 6379), admin panels, and internal APIs. The proxy correctly forwards Host, X-Forwarded-For, X-Forwarded-Proto, and X-Real-IP headers, which is good practice for upstream service compatibility.

4. Component Architecture Assessment

The component architecture follows a clean, flat structure where each major section of the page is a self-contained component. All custom components are "use client" components, which is necessary for Framer Motion animations and interactive state management. The data layer is centralized in skill-data.ts with well-defined TypeScript interfaces (Skill, DesignOption, SP7Dimension, WeightProfile, IntentRoute). This separation of data from presentation is a positive architectural pattern that makes the components testable and the data replaceable.

The 40 shadcn/ui components in `src/components/ui/` are standard library components that were auto-generated and are correctly configured with the new-york style variant. The `components.json` configuration properly maps aliases for components, utils, UI, lib, and hooks paths. The lucide icon library is specified as the icon source, which is consistent with the shadcn/ui ecosystem. The utility function `cn()` in `lib/utils.ts` correctly combines `clsx` and `tailwind-merge` for conditional class composition, which is the recommended pattern for shadcn/ui projects.

One notable architectural gap is the absence of React Error Boundaries. If any component throws during rendering (e.g., Recharts failing to render with empty data, or Framer Motion encountering an animation error), the entire page will unmount with a white screen. Adding error boundaries around each major section would provide graceful degradation and allow the rest of the page to function even if one section fails. Additionally, there are no loading states (Suspense boundaries) for any components, which would improve the perceived performance if components are later converted to dynamically imported or server components.

5. Security Deep Dive

5.1 Attack Surface Analysis

The application has a relatively small attack surface because it is primarily a static content site with no user authentication, no database writes from the client, and no server-side API endpoints beyond the placeholder `/api/route.ts`. However, the Caddy reverse proxy configuration (finding S01) significantly expands the attack surface by allowing external access to any localhost service. The WebSocket server (`examples/websocket/`) has its own attack surface through the CORS wildcard and unauthenticated `join/message` events, though this appears to be an example file rather than a production endpoint.

5.2 Client-Side Security

The `AIPortalGateway` search input performs client-side-only keyword matching, so there is no XSS or injection risk from the search functionality itself. However, the `navigator.clipboard` API usage in `SkillReference` and `ImplementationBlueprint` components will fail silently in non-HTTPS contexts, and the error is not caught or communicated to the user. The `VisualGallery` lightbox creates a full-screen overlay that can trap keyboard focus and has no Escape key handler, which is both a usability and accessibility issue. The application does not set any Content Security Policy (CSP) headers, which would be important if the application is deployed to production to prevent inline script injection and control resource loading origins.

5.3 Dependency Security

The `package.json` includes 35+ production dependencies, many using caret (^) version ranges which allow minor version updates that may introduce security vulnerabilities. The project uses React 19, which is relatively new and may have undiscovered vulnerabilities. The `z-ai-web-dev-sdk` package at version 0.0.17 is a very early release that may not have undergone security auditing. Running `npm audit`

(or bun audit) regularly and pinning critical dependencies to exact versions would improve the security posture. The next-auth package is included but not used, which means its code is in node_modules but never initialized, potentially exposing unused attack surface.

6. Prioritized Recommendations

Priority	Action	Findings	Effort
P0	Restrict Caddy proxy port whitelist; add auth middleware	S01	2-4 hours
P0	Restrict WebSocket CORS to allowed origins	S02	30 min
P1	Add try-catch to clipboard operations with fallback	S03	1 hour
P1	Add debounce to AI Portal search	S04	30 min
P2	Enable TypeScript strict mode and fix errors	B01,B03	4-8 hours
P2	Re-enable critical ESLint rules incrementally	B02	2-4 hours
P2	Normalize SP-7 weights; add maxScore guard	L01,L02	1 hour
P2	Fix lightbox AnimatePresence and editorial image	L03	1 hour
P3	Extract hardcoded colors to shared constants	B04,B05	2 hours
P3	Fix scroll offset for sticky navbar	L05	30 min
P3	Fix social metadata mismatch	B07	15 min
P3	Add Escape key handler to lightbox	E02	30 min
P4	Add mobile menu animation and refine hamburger	E03	1 hour
P4	Audit and remove unused npm dependencies	I01	2 hours

The P0 items should be addressed before any production deployment. The Caddy SSRF vulnerability and WebSocket CORS wildcard are exploitable security holes that could expose internal services or allow unauthorized access. P1 items are important for user experience and should be resolved in the next development sprint. P2 items improve code quality and prevent future bugs, and P3/P4 items are polish and maintenance tasks that can be scheduled for a future iteration. The total estimated effort to address all findings is approximately 18-28 hours of development work, with the security fixes (P0) requiring only 3-5 hours.

7. Additional Observations

7.1 Positive Patterns

The project demonstrates several positive architectural patterns worth highlighting. First, the centralized data model in skill-data.ts with TypeScript interfaces provides a single source of truth for all component data, making the application easy to modify and extend. Second, the consistent editorial design language (Ink and Vermillion palette, Georgia serif headings, horizontal rules, pull quotes) is applied uniformly

across all 11 sections, creating a cohesive visual identity. Third, the component decomposition follows a logical pattern where each section is a self-contained unit with its own state management, making the components independently testable and replaceable. Fourth, the use of framer-motion for scroll-triggered animations (`whileInView`) with `viewport={{ once: true }}` prevents unnecessary re-animation on scroll, which is good for performance. The `PrismaClient` singleton pattern in `db.ts` correctly avoids creating multiple database connections in development.

7.2 Performance Considerations

The application imports `framer-motion` and `recharts` as client-side dependencies, which add significant bundle weight. `Framer Motion` adds approximately 30KB gzipped and `Recharts` adds approximately 70KB gzipped to the client bundle. Since the page is a single long-scroll layout with 11 sections, all components load on initial page render. Consider implementing dynamic imports with `next/dynamic` for below-the-fold sections (sections 5-11) to reduce the initial JavaScript payload. The `SkillReference` component renders 16 items with individual `framer-motion` wrappers, which could be optimized using a stagger container pattern instead of individual `whileInView` triggers. The heatmap table renders 16 rows times 12 columns of interactive cells, which is manageable but should be monitored if the data grows.

7.3 Accessibility Gaps

The application has several accessibility gaps that should be addressed for WCAG 2.1 AA compliance. The navigation buttons lack `aria-label` attributes, making it difficult for screen reader users to understand the navigation structure. The lightbox in `VisualGallery` does not trap focus or provide a keyboard-close mechanism (Escape key). The color contrast in the heatmap visualization uses opacity-based coloring that may not meet the minimum 4.5:1 contrast ratio for text over background, particularly for cells with values 1-2 which display text in `#6B6B6B` over a near-transparent background. The editorial-dropcap CSS pseudo-element uses `float:left` which can disrupt the reading order for assistive technologies. The mobile hamburger menu button lacks an `aria-expanded` attribute to communicate its state. These issues are common in visually-driven projects but should be addressed for inclusive design compliance.